

CS 699 – Lec 2

Shell Scripting - BASH

Om Damani, CSE IIT Bombay

Logistics

- Computer Access
- TA office hours
- Default submission deadline: Tue 11.30pm even if moodle page says otherwise. If it is different from default, I will post a message.
- Storage
 - your labuser account is temporary one. files not saved
 - either copy on a memory stick
 - or, your permanent account is on mars - ssh there
 - or mail yourself – not desirable

Today's Class: Shell Scripting

- A shell script is a text file containing one or more shell commands that are executed sequentially by a shell interpreter.
- In technical world – a sequence of instructions
- scripts are interpreted – not compiled (like Java, C++), meaning earlier instructions are executed before reaching an erroneous instruction and aborting

Why scripting?

- Instead of typing commands one by one, collect a sequence of commands in one place so that you can execute them repeatedly without typing them every time.

Motivation

- Useful for automating your daily tasks on your personal computer or your research data management, code management etc. or any other automatic processing
- Example: Identify all duplicate files in a directory
- Goal: to show that most such tasks can be done quickly with sufficient knowledge of shell scripting

Creating a Shell Scripting file using text editor and Command Line

Create a script file using text editor.

- Open a new file in a text editor
- Add the Bash interpreter path at the beginning of the file, eg : `#!/bin/bash`
- Type the shell commands you want to execute
- Optional: Save the file with a `.sh` extension (e.g., `script.sh`).

Example Code:

```
#!/bin/bash

cd ~/cs699

echo "Number of files:"

ls | wc -l

echo "Total disk usage:"

du -hs

echo "Detailed file listing:"

ls -rtl | tail -n +2
```

Running into shell script execution error

- A newly created shell script is treated as a **regular text file**, It **does not have execute permission** by default
- Executing the script directly from the terminal will give permission error:
 - bash: permission denied: ./test_cs699.sh
- Make the script executable, by **Adding Execute Permission**:
 - **Add Execute Permission:** **chmod +x filename.sh**

Executing a Shell Script:

- **Example:**

```
ls -l ./test_cs699.sh
```

```
chmod +x ./test_cs699.sh
```

```
chmod 744 ./test_cs699.sh
```

The permission bits: **-rwxr-xr-x**

- **+x** – Adds execute (run) permission to the file.

- **Verifies the Permission:**

- **ls -l test_cs699.sh**

Why ./?

- **.** represents the **current directory**
- **./script_name.sh** tells the shell to execute the script located in the current directory
- Execute the script from the current directory

Special Arguments

\$0

Name of the current script

\$1, \$2, \$3
arguments)

Positional parameters (command-line

\$#

Number of command-line arguments

\$*
string

All command-line arguments as a single

\$@
strings

All command-line arguments as separate

\$\$

Process ID (PID) of the current shell/script

\$?
command

Exit status of the previously executed

!!
not a parameter or variable

repeats the previous cmd, It is

Parameter

- Parameters allow users to provide input when executing a script instead of hard-coding values inside the script
- **Why Use Parameters?** : Instead of editing the script every time, pass different values while executing it.
- Example: `./directory_info.sh ~/Downloads`
`./directory_info.sh ~/Documents`
OR `./ping_server.sh google.com`
`./ping_server.sh cse.iitb.ac.in`

Parameter : Command-Line argument

- Open a test_cs699.sh file and add..

```
#!/bin/bash
```

```
directory="$1"
```

```
echo "Directory: $directory"
```

```
echo "Number of files:"  
ls "$directory" | wc -l
```

```
echo "Disk usage:"  
du -sh "$directory"
```

```
echo "Detailed file listing:"  
ls -rtl "$1" | tail -n +2
```

Execution of script:

- Change mod to execute
- Example:
./filename.sh folder_name

Execute syntax:

```
./test_cs699.sh backup/
```

Functions in shell

- A **function** is a named block of commands that performs a specific task. Once defined, it can be called **multiple times** from anywhere in the script
- Organizes the script into **reusable code blocks** and makes scripts **easier to read, test, and maintain**
- **Syntax:** **function_name() { Commands }**

Example:

```
#!/bin/bash
greet() {
    echo "Welcome to CS699"
}
greet
greet
greet
```

Parameter : Function argument

- Open a test_cs699.sh file and add ..

```
#!/bin/bash
```

```
# Using a function parameter by storing it in a local variable
```

```
greet() {  
    message="$1"           # Stores function parameter in a variable
```

```
    echo "Message: $message"    # Reuse of the variable  
    echo "Length: ${#message}"  #length of parameter assigned  
}
```

```
greet "Good day!"           #Argument passed
```

Execution of script:

- Change mod to execute
- Example: ./filename.sh
"parameter"
- Execute syntax:
./test_cs699.sh

for parameter not used

When is \$n not-substituted :

- commands in your script may want to use \$n of their own
- for example:
 - `echo a b c | awk '{print $3 $1}'`
- add space:
 - `echo a b c | awk '{print $3} "$1}'`

Note: Ensure that the script is in parent directory and the target directory is the child directory

Complete code block:

```
#!/bin/bash  
target=$1
```

```
echo "Number of files in $target"  
ls "$target" | wc -l
```

```
echo "Total disk usage of $target"  
du -sh "$target" | awk '{print $1}'
```

```
echo "Detailed file listing of $target"  
ls -rtl "$target" | tail -n +2
```

Command Substitution:\$(command)

- Can you guess what `$()` does?
 - Executes the command inside the parentheses and replace it with its output.

Note: Ensure that the script is in parent directory

Complete code block:

```
#!/bin/bash
```

```
bck_dir="/Users/sakina/Desktop/cs_699/backup"
```

```
src_dir="/Users/sakina/Desktop/cs_699/source"
```

```
timestamp=$(date +"%Y%m%d%H%M%S")
```

```
tar -czf "$bck_dir/backup_$timestamp.tar.gz"  
"$src_dir"
```

Using Variables in Shell Scripts like other PL:

Using Variables in Shell Scripts allows to store values (such as file names or folder names) and reuse them throughout the script.

This makes the script easier to read and maintain.

Complete code block:

```
#!/bin/bash
```

```
# Variable Declaration and Assignment
```

```
project="cs699"  
datafile="data.csv"
```

```
# Display the variable values
```

```
echo "project directory: $project"  
echo "data file: $datafile"
```

```
find "$project" -type f | wc -l
```

```
#disk usage  
du -sh "$project"
```

```
#list of file in reverse, skip 1st line
```

```
ls -lrt "$project" | tail -n +2
```

Decision Making in Shell Scripts – **if,then,else**

- The **if-then-else** statement is used to make decisions
- It evaluates a condition and executes different blocks of code depending on whether the condition is **true** or **false**
- Example:
- Stores the student's marks in a variable
- **-ge** → Tests whether the first value is **greater than or equal to** the second value

Complete code block:

```
#!/bin/bash

marks=75

if [ "$marks" -ge 40 ]
then
    echo "Pass"
else
    echo "Fail"
fi
```


Example: Log File Rotation

A shell script that accepts a **log file** as a command-line argument.

- Checks the log file exists and file size is **greater than 1 MB**.
- Otherwise, display that **log rotation is not required**.
- Both `[ ]` and `[[ ]]` can be used to test conditions in an `if` statement
 - `[ ]` – Basic test command (POSIX compliant)
 - `[[ ]]` – Extended Bash test with additional features

Complete code block:

```
#!/bin/bash
```

```
log_file=$1  
max_size=1000000    # 1 MB
```

```
# Check whether the log file exists
```

```
if [ ! -e "$log_file" ]  
then  
    echo "$log_file does not exist."
```

```
# Check whether the file size exceeds 1 MB
```

```
elif [ "$(wc -c < "$log_file")" -gt "$max_size" ]  
then  
    mv "$log_file" "$log_file.old"  
    touch "$log_file"  
    echo "log rotated successfully."
```

```
else  
    echo "$log_file is not large enough to rotate."  
fi
```

Another Example: Creating a Test File of a Specific Size

- To try the log-rotation script, you need a file of certain size
- Searching “how to write a file of certain size”: please check it yourself

<https://www.tutorialspoint.com/create-a-file-of-certain-size-in-linux>

Example:

```
head --bytes 300000 /dev/zero > file3.txt
```

```
truncate -s 10485760 file.txt
```

- However, some of the commands given there do not work :

```
tail --bytes 1G /dev/zero > file4.txt
```

- even the head command given there needed some change

Repeating Tasks: **for** Loop

Suppose you are responsible for monitoring multiple servers.

Instead of running the **ping** command separately for each server, a **for loop** can automatically repeat the same task for every server in a list.

- Example: a shell script that checks whether a server is reachable using the **ping** command.
- **@** represents **all elements** of the array.

Complete code block:

```
#!/bin/bash

servers=("cse.iitb.ac.in" "iitb.ac.in"
"asc.iitb.ac.in")

# Repeat the same task for every server
for server in "${servers[@]}"
do
    if ping -c 1 "$server" &> /dev/null
    then
        echo "Server $server is reachable"
    else
        echo "Server $server is unreachable"
        # Send alert here
    fi
done
```

While Loop: For repeating tasks

- A **while** loop repeatedly executes a block of commands **as long as a specified condition is true**

- **Example1:**

```
#!/bin/bash
```

```
x=1
```

```
while [ $x -le 3 ]
```

```
do
```

```
    echo "Line $x"
```

```
    x=$((x + 1))
```

```
done
```

Example2:

```
#!/bin/bash
```

```
while sleep 10
```

```
do
```

```
    df -h | grep "/dev/disk3s2" | awk  
'{print $5}' | tr -d '%'
```

```
done
```

- Every **10 seconds**, it checks the disk usage of the filesystem **/dev/disk3s2 (disk partitions)** and prints the percentage of disk space currently used.

Practice: Automate a Task

A shell script to automate repetitive tasks by combining simple Linux commands into a reusable program.

Analyse the data file from last class and display department-wise pass-fail count

Complete code block:

```
#!/bin/bash

FILE="data.csv"

echo "Checking file..."

if [ -f "$FILE" ]
then
    echo "$FILE found."
else
    echo "$FILE not found."
    exit 1
fi

echo "Total number of lines:"
wc -l "$FILE"

echo "Number of Pass students:"
grep -c "Pass" "$FILE"

echo "Number of Fail students:"
grep -c "Fail" "$FILE"

echo "Departments"

for dept in CSE EE ME
do
    echo "Department: $dept"
    grep "$dept" "$FILE"
    echo ""
done

greet() {
    echo "Analysis completed by $1"
}

greet "Studentname"
```

Practice: Switch case

- Reading user input using `read`
- Making decisions using a `case` statement
- `-n` means “do not print a newline”
- `1` expects only one character

Complete code block:

```
#!/bin/bash
```

```
echo -n "Please type in your name: "  
read name  
echo "Your name is ${name}"
```

```
echo -n "Please type in your gender [M/F/T]: "  
read -n 1 gender
```

```
case "$gender" in [mM])  
    echo -e "\nYour answer: Male";;  
[fF])  
    echo -e "\nYour answer: Female";;  
[tT])  
    echo -e "\nYour answer: Transgender";;  
*)  
    echo -e "\nInvalid choice: $gender";;  
esac
```

Practice: Basic System Information Report

- A shell script that displays the following system information:
 - Date
 - Current logged-in user
 - Current working directory

Practice: Copy a File to a Remote Server (**scp**)

- A shell script that copies a file named **notes.txt** to a remote server using **scp**
 - Syntax: **scp <source> <destination>**
- Where on the remote server is the file copied?
- Syntax: **"\$SERVER:~/**"
 - **~/** means the home directory of the remote user

Note: The file must exist in your current working directory, unless you specify a full path.

- eg: **FILE="/Users/sak/Documents/notes.txt"**

Practice: Display Current Login Information

- A a shell script that displays:
 - The current logged-in user
 - All users currently logged into the system
 - The total number of logged-in users.

Assignment questions:

Question 1: Shell Script

Write a Bash script named `project_summary.sh` that performs the following tasks:

- Stores the project directory name in a variable.
- Checks whether the directory exists.
- Displays the total number of files inside the project.
- Displays the total disk usage of the project.
- Lists all files sorted by modification time.
- Prints "`Analysis Completed`" after successful execution.

Submit both the shell script and its output.

Question 2 – Shell Script

Write a Bash script named `analyze.sh`.

The script should:

- Check whether `students.csv` exists.
- If the file does not exist, display an error message and terminate.
- If the file exists:
 - Display the total number of lines.
 - Display the number of students belonging to the CSE department.
 - Display the number of students belonging to the EE department.
 - Print "`Analysis Completed`" at the end.

Question 3 — Find the Largest Directories

Write a script to display the **10 largest directories** inside your home directory, sorted from largest to smallest and:

- Display directory size in human-readable format.
- Ignore permission denied errors.
- Display only the largest 10 entries.

Question 4 — Duplicate File Names

Suppose your project contains thousands of files spread across many subdirectories.

- Write commands to display **all filenames that occur more than once**, regardless of their location.

Question 5 — Directory Health Report

Write a shell script that accepts multiple directory names as input.

For each directory:

- Verify whether it exists.
- Count the number of files.
- Display disk usage.
- Print whether the directory is Small (<100 MB), Medium (100 MB–1 GB) or Large (>1 GB).

Use:

- functions
- loops
- if-else conditions

Question 6 — System Programming using Shell Commands

System Information Report - Write a shell script that generates a report containing:

- Current date and time
- Current logged-in user
- Hostname
- Current working directory
- Available disk space
- Available memory
- System uptime

Save the report in a text file

Question 7 — User Login Report

Write a shell script that displays:

- All users currently logged in
- Login time of each user
- Number of logged-in users
- Whether a specified user (passed as an argument) is currently logged in
- The last 10 user logins
- Number of unique users who logged in
- The most recent logins

Save the report in a text file

Question 8 —

Write a shell script that displays:

- All users currently logged in
- Login time of each user
- Number of logged-in users
- Whether a specified user (passed as an argument) is currently logged in
- The last 10 user logins
- Number of unique users who logged in
- The most recent logins

Save the report in a text file

Question 9 — Process Status and Process Management

Write a shell script that accepts a process name.

The script should:

- Check whether the process is running.
- Display its Process ID (PID).
- Display CPU usage.
- Display memory usage.
- Print an appropriate message if the process is not found.

Save the report in a text file